Extend the existing Selenium-based generation logic to support Playwright.

Wherever Selenium code, locators, page objects, or step definitions are currently generated, implement equivalent Playwright-based generation using TypeScript.

Requirements:

- Maintain the SAME functionality, structure, and behavior as Selenium generation.
- Replace Selenium WebDriver logic with Playwright APIs.
- Use Playwright locator strategy:
    - Prefer getByRole, getByLabel, getByPlaceholder, getByText
    - Avoid XPath unless absolutely necessary
- Generate:
    - Playwright Page Object Models (TypeScript)
    - Playwright test scripts or Cucumber step definitions (TypeScript)
    - Feature files remain unchanged
- Follow Playwright best practices:
    - async/await usage
    - built-in auto-waiting (no explicit waits like Selenium)
- Keep output clean, structured, and code-only (no explanations)

Input will include DOM content and user-selected elements, same as Selenium flow.

Output should mirror Selenium output format, but using Playwright syntax and standards.

Implemented end-to-end Playwright TypeScript generation in the same architecture and flow as Selenium generation.

Changes made:

1. Added Playwright TypeScript page-object prompt template

- Added PLAYWRIGHT_TYPESCRIPT_PAGE_ONLY with:
    - TypeScript Page Object output
    - Playwright async/await usage
    - Locator preference order: getByRole, getByLabel, getByPlaceholder, getByText
    - XPath avoidance guidance
    - Code-only output constraint
- See [prompts.js:48]

2. Added Playwright TypeScript Cucumber step-definition prompt template

- Added CUCUMBER_WITH_PLAYWRIGHT_TYPESCRIPT_STEPS with:
    - Unchanged feature-file generation requirement
    - TypeScript Playwright step definitions
    - Built-in auto-waiting guidance (no explicit waits/sleeps)
    - Same combined output shape as Selenium flow (gherkin + code block)
    - Code-only output constraint
- See prompts.js:241

3. Extended prompt routing logic to support TypeScript + Playwright

- Updated generation decision tree so:
    - Page only -> PLAYWRIGHT_TYPESCRIPT_PAGE_ONLY
    - Feature + page -> CUCUMBER_WITH_PLAYWRIGHT_TYPESCRIPT_STEPS
    - Same fallback behavior retained for unsupported combos
- See chat.js:555
- New helper added at chat.js:616

4. Updated support message text

- Unsupported combination message now lists both supported paths:
    - Java/Selenium
    - TypeScript/Playwright
- See chat.js:636

5. Aligned loader behavior with selected generation path

- Set codeGeneratorType from resolved prompt key before generation, so Playwright runs display Playwright loader label correctly
- See chat.js:377

6. Added generator type constants for Playwright

- Extended CODE_GENERATOR_TYPES for new Playwright prompt keys
- See prompts.js:353

Validation:

1. No syntax/errors reported in changed files via diagnostics for:

- prompts.js
- chat.js

```
4   export const DEFAULT_PROMPTS = {
47    */
48    PLAYWRIGHT_TYPESCRIPT_PAGE_ONLY: `
49      Instructions:
50      - Generate ONLY a Playwright TypeScript Page Object Class (no test code).
51      - Add TypeScript doc comments for class and methods.
52      - Use Playwright APIs with async/await.
53      - Use meaningful method names.
54      - Prefer Playwright locators in this order: getByRole, getByLabel, getByPlaceholder, getByText.
55      - Avoid XPath unless absolutely necessary.
56      - Do NOT include explanations or test code.
57
58      Context:
59      DOM:
60      \`\`\`html
61      \${domContent}
62      \`\`\`
63
64      Example:
65      \`\`\`typescript
66      import { Page, Locator } from '@playwright/test';
67
68      /**
69       * Page Object for Component Page
70       */
71      export class ComponentPage {
72        readonly page: Page;
73
74        constructor(page: Page) {
75          this.page = page;
76        }
77
78        // Add methods as per the DOM
79      }
80      \`\`\`
81
82      Persona:
83      - Audience: Automation engineer focusing on maintainable POM structure.
84
85      Output Format:
86      - A single TypeScript class inside a \`\`\`typescript\`\`\` block.
87      - Code only. No explanations.
88
89      Tone:
90      - Clean, maintainable, enterprise-ready.
91    `,
92
93    /**
```

/**

* Cucumber with Playwright TypeScript Step Definitions

*/

CUCUMBER_WITH_PLAYWRIGHT_TYPESCRIPT_STEPS: `

Instructions:

- Generate BOTH:

  1. A Cucumber .feature file.

  2. A TypeScript step definition file for Playwright.

- Do NOT include Page Object code.

- Feature file structure should remain equivalent to Selenium generation.

- Step defs must use Playwright APIs with async/await and built-in auto-waiting.

- Do NOT use explicit waits or sleep utilities.

- Prefer Playwright locators in this order: getByRole, getByLabel, getByPlaceholder, getByText.
- Avoid XPath unless absolutely necessary.
- Use Scenario Outline with Examples table (South India realistic data).

Context:

DOM:

\`\`\`html

\${domContent}

\`\`\`

URL: \${pageUrl}

Example:

\`\`\`gherkin

Feature: Login to OpenTaps

Scenario Outline: Successful login with valid credentials

  Given I open the login page

  When I type "<username>" into the Username field

  And I type "<password>" into the Password field

  And I click the Login button

  Then I should be logged in successfully

Examples:

  | username | password |

  | admin | admin123 |

\`\`\`

\`\`\`typescript

import { Given, When, Then, Before, After } from '@cucumber/cucumber';

import { Browser, BrowserContext, Page, chromium, expect } from '@playwright/test';

```
let browser: Browser;
let context: BrowserContext;
let page: Page;

Before(async function () {
  browser = await chromium.launch({ headless: false });
  context = await browser.newContext();
  page = await context.newPage();
});

After(async function () {
  await context?.close();
  await browser?.close();
});

Given('I open the login page', async function () {
  await page.goto('\${pageUrl}');
});

When('I type {string} into the Username field', async function (username: string) {
  await page.getByLabel('Username').fill(username);
});

When('I type {string} into the Password field', async function (password: string) {
  await page.getByLabel('Password').fill(password);
});

When('I click the Login button', async function () {
  await page.getByRole('button', { name: 'Login' }).click();
});
```

Then('I should be logged in successfully', async function () {

  await expect(page.getByText('success')).toBeVisible();

});

\`\`\`

Persona:

- Audience: QA engineers working with Cucumber & Playwright.

Output Format:

- Gherkin in \`\`\`gherkin\`\`\` block + TypeScript code in \`\`\`typescript\`\`\` block.

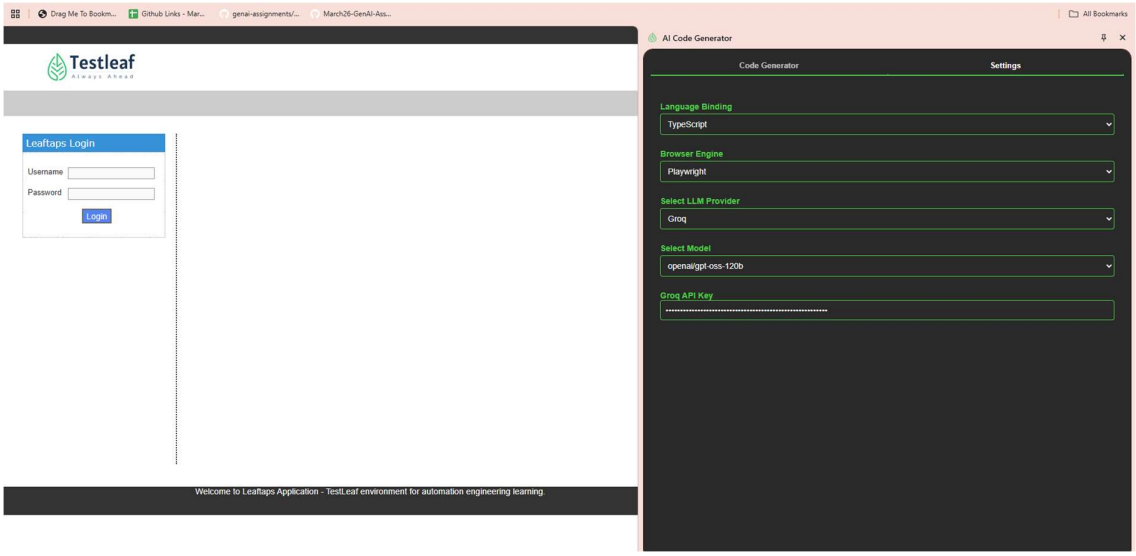
- Code only. No explanations.

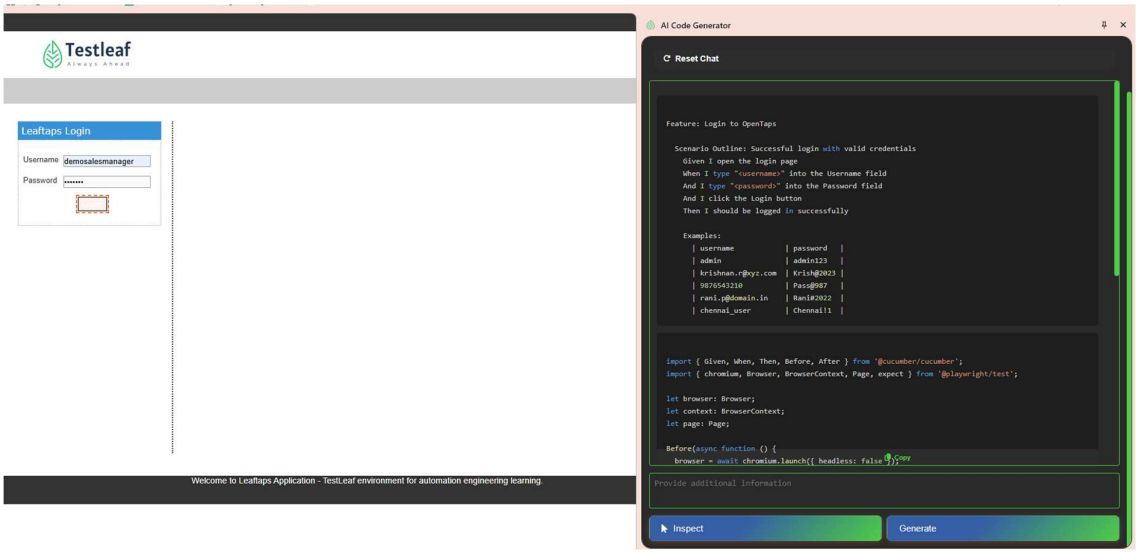
Tone:

- Professional, executable, structured.

`

};





Feature: Login to OpenTaps

Scenario Outline: Successful login with valid credentials

Given I open the login page

When I type "<username>" into the Username field

And I type "<password>" into the Password field

And I click the Login button

Then I should be logged in successfully

Examples:

| username | password |

| admin | admin123 |

| krishnan.r@xyz.com | Krish@2023 |

| 9876543210 | Pass@987 |

| rani.p@domain.in | Rani#2022 |

| chennai_user | Chennai!1 |

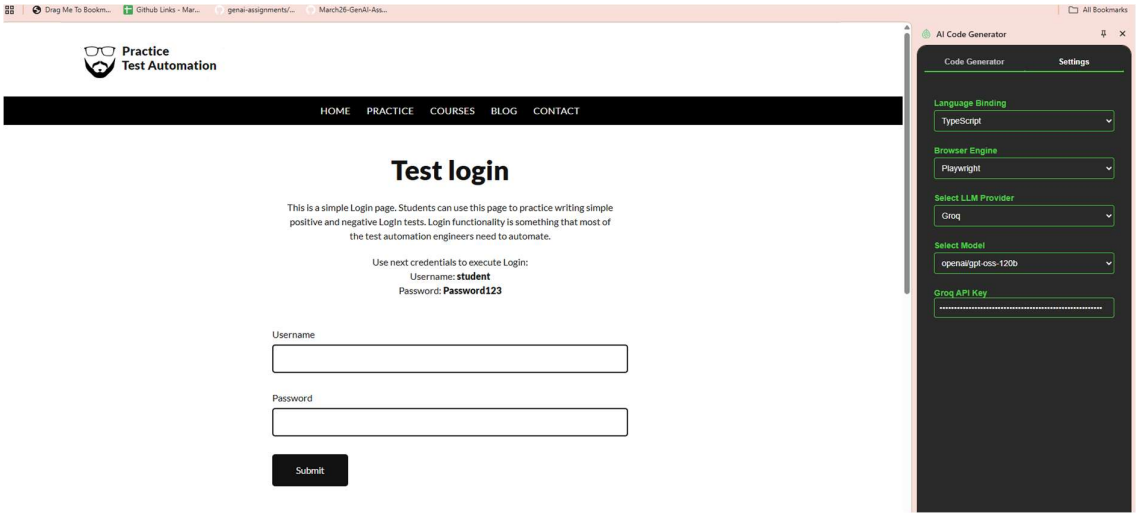
import { Given, When, Then, Before, After } from '@cucumber/cucumber';

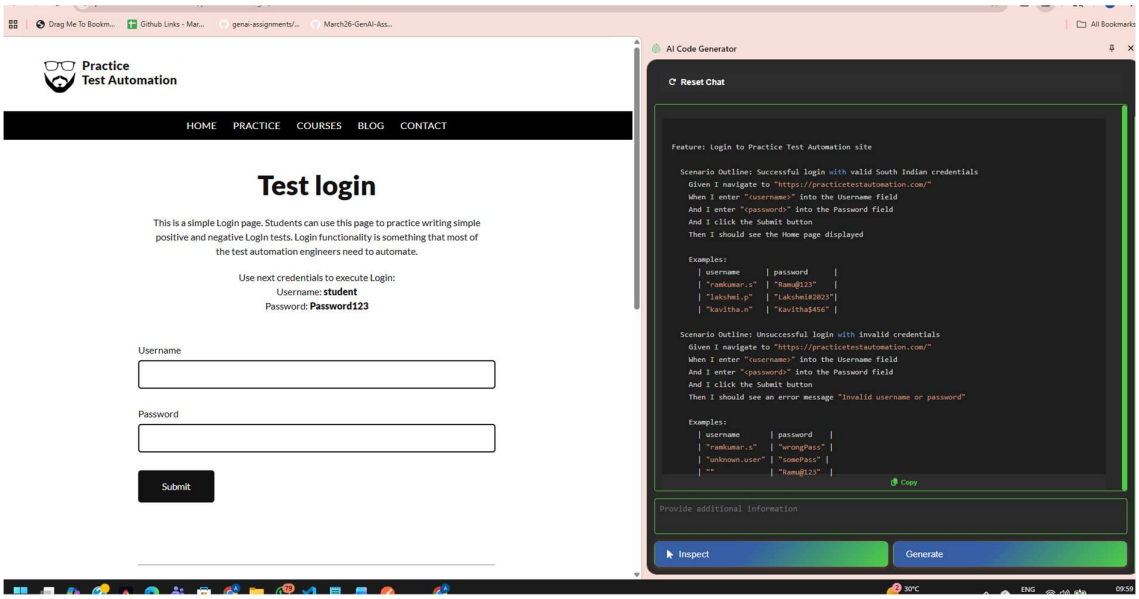
import { chromium, Browser, BrowserContext, Page, expect } from '@playwright/test';

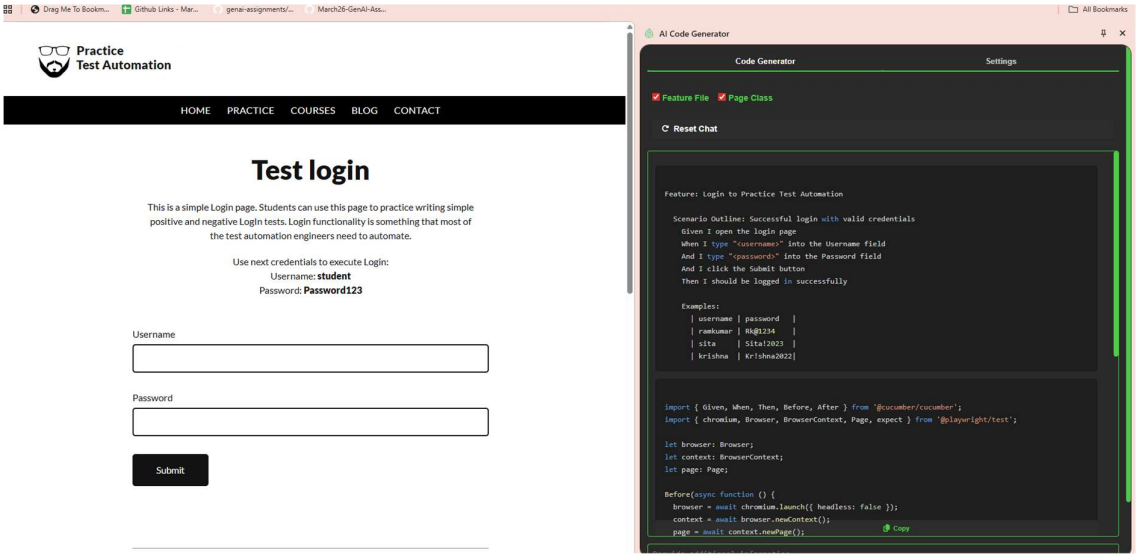
let browser: Browser;

```
let context: BrowserContext;
let page: Page;

Before(async function () {
  browser = await chromium.launch({ headless: false });
  context = await browser.newContext();
  page = await context.newPage();
});

After(async function () {
  await context?.close();
  await browser?.close();
});

Given('I open the login page', async function () {
  await page.goto('https://leaftaps.com/opentaps/control/main');
});

When('I type {string} into the Username field', async function (username: string) {
  // No visible label/placeholder, using the element's id
  await page.locator('#username').fill(username);
});

When('I type {string} into the Password field', async function (password: string) {
  await page.locator('#password').fill(password);
});

When('I click the Login button', async function () {
  await page.getByRole('button', { name: /login/i }).click();
});
```

```
Then('I should be logged in successfully', async function () {
  await expect(page.getByText(/welcome|home|dashboard/i)).toBeVisible();
});
```







Feature: Login to Practice Test Automation

Scenario Outline: Successful login with valid credentials

Given I open the login page

When I type "<username>" into the Username field

And I type "<password>" into the Password field

And I click the Submit button

Then I should be logged in successfully

Examples:

| username | password |

| ramkumar | Rk@1234 |

| sita | Sita!2023 |

| krishna | Kr!shna2022|

```
import { Given, When, Then, Before, After } from '@cucumber/cucumber';
import { chromium, Browser, BrowserContext, Page, expect } from '@playwright/test';

let browser: Browser;
let context: BrowserContext;
```

```
let page: Page;

Before(async function () {
  browser = await chromium.launch({ headless: false });
  context = await browser.newContext();
  page = await context.newPage();
});

After(async function () {
  await context?.close();
  await browser?.close();
});

Given('I open the login page', async function () {
  await page.goto('https://practicetestautomation.com/practice-test-login/');
});

When('I type {string} into the Username field', async function (username: string) {
  await page.locator('#username').fill(username);
});

When('I type {string} into the Password field', async function (password: string) {
  await page.locator('#password').fill(password);
});

When('I click the Submit button', async function () {
  await page.getByRole('button', { name: 'Submit' }).click();
});

Then('I should be logged in successfully', async function () {
  await expect(page.getByText('Login Successful')).toBeVisible();
```

```
});
```